

Using Gradle in CS F213 Lab

Aman Pathak, PhD TA

`p20260018@goa.bits-pilani.ac.in`

CS F213 Lab

The problem: building by hand does not scale

For one file, this is fine:

```
javac Calculator.java  
java Calculator
```

Real projects break this habit quickly:

- many files in packages, compiled in the right order
- third-party libraries (where do the jars live?)
- tests that must run the same way on every machine
- “it works on my laptop” is not a build strategy

A build tool takes over all of this. Ours is Gradle, and we drive it from the terminal.

Why Gradle: what you get

You describe *what* the project is, Gradle figures out *how* to build it:

- **One setup command:** `gradle init` scaffolds the whole project, folders and build files included
- **Dependencies by name:** libraries fetched from Maven Central, no jars passed around by hand
- **Standard tasks:** `build`, `test`, `run`, `clean` work the same in every project
- **Incremental builds:** only what changed gets rebuilt
- **Same build everywhere:** the wrapper pins one Gradle version for the whole class

gradle vs. gradlew: the one distinction to remember

	gradle	./gradlew
What it is	system install	wrapper script inside the repo
Version	whatever that machine has	pinned in gradle/wrapper/*.properties
Comes from	pre-installed	created by gradle init

Rule for this course: set the project up with gradle, then do everything else with ./gradlew. Same command, same version, every machine:

```
gradle init      # once: scaffolds the project (needs system Gradle)
./gradlew build  # always: builds with the pinned version
```

Both commands must be run **inside the project root** (the folder that holds settings.gradle.kts).

Further reading: [Gradle Wrapper](#) | [Build Init plugin](#)

Step 0: check your tools

```
java -version      # we use Java 21
gradle --version   # we use Gradle 9.x
```

You need exactly two VS Code extensions, nothing more:

- Microsoft Extension Pack for Java (language support, debugger, test runner; Red Hat support is already inside it)
- Gradle for Java (the Gradle tasks panel)

Do not use Code Runner: it compiles single files directly and ignores the build files and the test framework.

Step 1: create the project with gradle init

Make an empty folder, step into it, and run one command:

```
mkdir gradle-demo  
cd gradle-demo  
gradle init
```

Gradle now asks a short series of questions. Answer them exactly like this (plain Enter means “take the default”):

Question	Your answer
Type of build	1: Application
Implementation language	1: Java
Target Java version	Enter (default: 21)
Project name	Enter (default: folder name)
Application structure	1: Single application project
Build script DSL	1: Kotlin (or Enter, it is the default)
Test framework	1: JUnit 4
New APIs and behavior	Enter (default: no)

Step 2: look at what init just made

You wrote no files yourself. The project now looks like this:

```
gradle-demo/  
  settings.gradle.kts  
  gradlew / gradlew.bat          <- the wrapper (use this from now on)  
  gradle/wrapper/                <- pins the Gradle version  
  app/build.gradle.kts          <- the build file init wrote for you  
  app/src/main/java/org/example/App.java  
  app/src/test/java/org/example/AppTest.java
```

Two things to notice:

- the wrapper (gradlew) is part of the project now, so from here on every command starts with ./gradlew
- tests already live where they belong: src/test/java/..., next to a sample JUnit 4 test

Step 2b: reading the file `init` wrote

`app/build.gradle.kts`, trimmed to what matters:

```
plugins {  
    application           // brings the run task (and java tasks)  
}  
  
repositories {  
    mavenCentral()       // where libraries are fetched from  
}  
  
dependencies {  
    testImplementation(libs.junit) // JUnit 4, version lives  
                                    // in gradle/libs.versions.toml  
}  
  
application {  
    mainClass = "org.example.App" // what ./gradlew run starts  
}
```

Kotlin DSL notes: function-call style with double quotes (`mavenCentral()`), and versions kept in one shared catalog file instead of being scattered through the build.

Step 3: tasks come free with the plugins

You never define `build`, `test`, `clean` or `run`. The generated build file applies the right plugins, and the plugins add the tasks:

- the `java` plugin adds `build`, `test`, `clean`, `check`, `jar`, ...
- the `application` plugin adds `run`

Prove it to yourself, from the project root:

```
./gradlew tasks --all
```

Step 4: the terminal workflow (1 of 2)

From the project root:

```
./gradlew build    # compile + test + jar, the full check  
./gradlew test     # only the tests  
./gradlew clean    # delete the build/ folder
```

A healthy build ends with:

```
> Task :compileJava  
> Task :test  
> Task :build  
BUILD SUCCESSFUL
```

Step 4: the terminal workflow (2 of 2)

```
./gradlew run           # run the app, prints Hello World!
./gradlew test --rerun-tasks # run tests again even if
                           # nothing changed
./gradlew test --info    # extra detail when hunting
                           # a failure
```

Reports land under build/ (git-ignored), for example
app/build/reports/tests/test/index.html.

Adding production code: the rules

init gave you a working sample. When you add your own classes:

- ❶ Put the file under `app/src/main/java/`, in the folder that matches its package.
- ❷ One public class per file; the file name matches the class name.
- ❸ Only code with a `main()` method can be *run*. Helper classes are used through tests or through the app, never run directly.

```
package org.example;

public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("divisor must not be 0");
        }
        return a / b;
    }
}
```

Adding tests: the rules (JUnit 4)

- ❶ Put the file under `app/src/test/java/`, in the same package folders as the code it tests.
- ❷ Name it after the class under test plus `Test` (`Calculator` becomes `CalculatorTest`).
- ❸ JUnit 4 shape: import `org.junit.Test`, annotate each test method with `@Test`, assert with `org.junit.Assert`. Test methods are `public void`.

```
package org.example;

import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class CalculatorTest {

    private final Calculator calc = new Calculator();

    @Test
    public void addsTwoNumbers() {
        assertEquals(5, calc.add(2, 3));
    }

    @Test(expected = IllegalArgumentException.class)
    public void divideByZeroThrows() {
        calc.divide(1, 0);
    }
}
```

References: the original Gradle docs

Everything in this deck comes from these pages. When in doubt, read the docs before asking:

- [Build Init plugin](#): every `gradle init` prompt explained
- [Gradle Wrapper](#): how the pinned version works
- [Building Java projects](#): plugins, source layout, tasks
- [Testing in Gradle](#): how `test` finds and runs JUnit
- [Beginner tutorial: Initializing the Project](#): the full init-generated Java example

Cheat sheet (take a photo)

```
mkdir gradle-demo && cd gradle-demo
gradle init          # 1 Application, 1 Java, 21,
                    # 1 Single, 1 Kotlin, 1 JUnit 4

./gradlew build      # full check
./gradlew test       # tests only
./gradlew run        # run the app
./gradlew clean      # start fresh
```

Stuck? Ask in this order: read the terminal output, open the HTML report under `app/build/reports/`, then ask the TA. Thank you!